

TP1 — Le réseau du métro de Montréal

Carnet, graphes, Dijkstra — et un premier modèle

Intelligence Artificielle 2 (420-316-AH)

Mise en situation

Vous sortez du Collège, station Crémazie. Votre téléphone vous propose un trajet, et vous vous demandez pourquoi *celui-là*. Derrière l'écran, il y a exactement ce que vous avez appris depuis la séance 2 : un graphe, un parcours, et un algorithme glouton.

Dans ce TP, vous rebâissez ce moteur vous-mêmes, dans un carnet Jupyter, sur les 68 stations du métro de Montréal. Puis vous vous servez de votre propre travail pour fabriquer les données d'un **premier modèle**.

Ce que ce TP évalue. Votre capacité à réutiliser, sur de vraies données, ce qui a été vu en classe : un carnet propre (séance 5), un graphe (séance 3), un parcours en largeur et son arbre (séance 3), Dijkstra (séance 4), et les bases du machine learning (séance 7). **Le code seul ne suffit pas : chaque question, même de programmation, se termine par une cellule d'observation à rédiger, et ces cellules plus la partie 8 valent à elles seules 25 points sur 100.**

Seul ou en équipe de 2. Une équipe remet **un seul carnet** et indique à la toute fin **qui a fait quoi**, en une ou deux phrases par personne. Les deux membres doivent avoir commité dans le dépôt.

1 Fichiers fournis

- `TP1_depart.ipynb` — le carnet à compléter. Vous ne remettez que celui-là.
- `donnees/metro_montreal.csv` — une ligne par lien entre deux stations voisines : `station_a`, `station_b`, `ligne`, `minutes`.
- `donnees/stations.csv` — une ligne par station : `station`, `ligne`, `correspondance`. Il sert surtout à la partie 6.

Les durées sont **approximatives** (environ 2 minutes par inter-station, 1 ou 3 là où la distance le justifie) et les correspondances sont gratuites. C'est un modèle simplifié : vous en parlerez à la dernière question.

2 Travail à faire

Partie 1 — votre carnet (identification)

(—)

Dans la première cellule Markdown, écrivez votre nom, celui de votre coéquipier s'il y a lieu, et votre groupe.

Partie 2 — charger et explorer le réseau**(6 points)**

- Chargez les deux CSV (le code vous est donné).
- Affichez le nombre de stations et le nombre de **liens**. Attention : chaque lien apparaît deux fois dans le dictionnaire.
- Trouvez la station qui a le plus de voisins, et la liste des terminus.
- Comptez combien de tronçons durent 1, 2 et 3 minutes.

Partie 3 — le parcours en largeur, et son arbre**(10 points)**

- Complétez `bfs(reseau, depart)`. Il rend `distance` (en nombre d'arrêts) et `precedent`.
- Lancez-le depuis la station de votre choix : combien de stations sont atteintes ? Laquelle est la plus loin ?
- Dessinez avec `graphviz` l'**arbre de parcours** limité aux 3 premiers niveaux.

Partie 4 — Dijkstra, en minutes**(12 points)**

Reprenez les fonctions de la séance 4 : `plus_petite_case`, `relacher`, `dijkstra`, puis `chemin` et `duree`. Affichez le trajet le plus rapide entre deux stations de votre choix, avec sa durée.

Partie 5 — comparer les deux, et surligner le chemin**(10 points)**

Écrivez `comparer(depart, arrivee)` qui affiche, pour chaque algorithme, le nombre d'arrêts et la durée. Testez-la sur **trois** paires, dont **au moins une** où les deux ne donnent pas le même trajet.

Puis, sur cette paire divergente, **surlignez le chemin de Dijkstra** dans un dessin `graphviz` : arêtes du chemin en rouge et en épais, durée de chaque tronçon en étiquette, et stations du chemin remplies d'une couleur.

Les attributs à utiliser — c'est le seul coup de pouce de cet énoncé. Sur une arête : `color="red"`, `penwidth="3"`, `fontcolor="red"`. Sur un sommet : `style="filled"`, `fillcolor="#ffd9d9"`. Le reste — quelles arêtes dessiner, comment reconnaître celles qui appartiennent au chemin — est à vous.

Partie 6 — construire un tableau de données**(8 points)**

On passe à la séance 7. Un modèle a besoin d'un tableau : une ligne par exemple, une colonne par donnée, et une colonne à part pour l'étiquette.

Ici, **un exemple = une station**, et l'étiquette est sa **ligne**. Les données, vous les fabriquez avec **votre propre travail des parties 3 et 4** : nombre de voisins, durée moyenne et durée maximale des tronçons, puis le nombre d'arrêts (*votre BFS*) et le nombre de minutes (*votre Dijkstra*) qui séparent la station d'un **repère**.

Comptez ensuite combien de stations portent chaque étiquette, et déduisez-en la note qu'obtiendrait un modèle qui répondrait **toujours la même chose**.

Partie 7 — entraîner un modèle, et le noter honnêtement**(11 points)**

- Coupez en deux paquets (70 / 30) avec `train_test_split`, entraînez un `DecisionTreeClassifier`, et affichez **deux** notes : sur les stations apprises, et sur les stations jamais vues.
- Refaites-le avec **2** puis **3** repères, et mettez les trois notes côte à côte.
- Sur vos meilleures mesures, comparez un **arbre seul** à une **forêt aléatoire**. Relancez chacun avec `random_state` valant 0, 1, 2, 3 puis 4, et affichez pour chaque modèle ses cinq notes, leur **moyenne** et l'**écart** entre la plus haute et la plus basse.

Partie 8 — vos conclusions

(15 points)

Dix questions, en Markdown, en phrases complètes, en français. Comptez deux ou trois phrases par question, et **citez vos propres chiffres**. Quatre portent sur le réseau, quatre sur le modèle, deux sur l'ensemble. Elles sont écrites dans le carnet.

Chaque question se termine par une observation. Y compris les questions de programmation : le carnet contient **16 cellules d'observation**, une après chaque question. Vous y écrivez, en phrases complètes, ce que vous constatez — pas ce que fait votre code, mais ce que le résultat vous apprend. Elles valent **6 points** en tout, et une cellule laissée telle quelle vaut 0 pour sa question.

3 Qualité du carnet

(10 points)

Votre carnet est un **rapport**, pas un brouillon. Dix points portent sur sa tenue générale.

Ce que je regarde, **sans m'y limiter** : qu'il s'exécute du début à la fin sans erreur ; qu'on s'y retrouve, question par question ; que le code soit lisible et débarrassé de vos essais ; que les sorties soient compréhensibles sans relire le code ; que le français soit relu ; et que l'identification et la remise soient complètes.

Ces points se perdent par accumulation. Quelques exemples de ce que je vois chaque session : des cellules exécutées dans le désordre, des variables nommées `x2` ou `data_final_v3`, un `print` qui affiche un nombre sans dire de quoi il s'agit, des cellules d'essai laissées en plein milieu, un dessin illisible, une observation expédiée en trois mots. Aucun de ces défauts n'est grave à lui seul ; ensemble, ils coûtent les dix points.

4 Utilisation de Git

(10 points)

Le dépôt fait partie de l'évaluation, pas seulement le carnet.

La nomenclature des branches et des messages de commit doit respecter ce qu'on a vu en cours. Je regarde aussi l'historique : un travail construit au fil du temps plutôt que déposé d'un seul coup, et, en équipe de deux, la contribution réelle des deux membres.

Un dépôt remis en un seul commit, ou dont les branches et les messages ne suivent pas la convention vue en classe, perd tout ou partie de ces dix points.

5 Utilisation de l'intelligence artificielle

L'IA est **permise pour apprendre**, pas pour produire votre travail à votre place.

Permis :

- lui demander d'expliquer une notion (ce qu'est un graphe pondéré, ce que fait `train_test_split...`);
- lui faire clarifier un message d'erreur que vous ne comprenez pas ;
- lui demander un exemple sur **d'autres données que les nôtres**, inventées par vous.

Interdit :

- **coller cet énoncé**, en tout ou en partie, dans une conversation avec une IA ;
- coller le contenu des fichiers de données ;
- lui faire écrire votre BFS, votre Dijkstra, vos observations ou vos conclusions.

Obligatoire. La dernière section du carnet est une **déclaration d'utilisation de l'IA**. Vous y indiquez quel outil vous avez utilisé, pour quelle question, ce que vous lui avez demandé, et vous **collez le lien de partage de chaque conversation**. Ces liens doivent être ouvrables au moment de la correction.

Même si vous n'avez rien utilisé, la déclaration doit être remplie : vous écrivez « non » et vous signez les trois confirmations. Une déclaration **absente ou fausse vaut 0 pour l'ensemble du TP**. Je peux vous demander d'expliquer à l'oral n'importe quelle ligne de votre carnet.

6 Règles

- Le BFS et Dijkstra doivent être **écrits par vous**. Aucune bibliothèque d'algorithmes de graphes (`networkx`, `scipy.sparse.csgraph...`).
- `graphviz` est permis, et attendu, **pour dessiner seulement**.
- `scikit-learn` est permis pour les parties 6 et 7 seulement, et uniquement ce qui a été vu en classe : `train_test_split`, `DecisionTreeClassifier`, `RandomForestClassifier`, `.fit`, `.score`, `.predict`.
- Avant de remettre : `Kernel > Redémarrer et tout exécuter`. Un carnet dont les crochets ne se lisent pas `[1] [2] [3]...` perd les points de qualité.

Plusieurs bonnes réponses sont possibles. Quand deux trajets ont le même nombre d'arrêts, le BFS garde celui qu'il a rencontré en premier — cela dépend de l'ordre de vos voisins. Votre trajet peut donc différer de celui de votre voisin de classe sans qu'aucun des deux ne soit faux. C'est justement le sujet de la question 4. Il en va de même pour le modèle : vos repères ne sont pas forcément les miens, donc vos notes non plus.

7 Remise

La remise se fait comme en séance 6, sur votre dépôt GitHub :

1. Créez une branche pour le TP. **Son nom suit la convention vue en cours.**
2. Commitez votre carnet au fur et à mesure, avec des messages qui suivent eux aussi la convention vue en cours. Si vous êtes deux, **les deux doivent apparaître** dans l'historique.
3. **Publish branch**, puis **Create Pull Request** avec une description de ce que vous avez fait.
4. Fusionnez la PR dans **main** et déposez le lien du dépôt sur Omnivox.

8 Barème

Critère	Points
Partie 2 — charger et explorer	6
Partie 3 — BFS, arbre de parcours et dessin	10
Partie 4 — Dijkstra, chemin et durée	12
Partie 5 — comparaison sur trois paires, dont une divergente, et surlignage	10
Partie 6 — tableau de données construit à partir de vos fonctions	8
Partie 7 — deux notes, trois repères, arbre contre forêt	11
Partie 8 — conclusions écrites (10 questions)	15
Les 16 cellules d'observation, une par question	6
Qualité du carnet (section 3)	10
Utilisation de Git (section 4)	10
Déclaration d'utilisation de l'IA, complète et vérifiable	2
Total	100